CachePilot — Implementation Document

Semantic Caching & Cost-Aware Model Routing for RAG

Version 1.0 | 27 October 2026 | Engineering specification for an independently buildable portfolio project

Document control: Treat this document as the project-specific source of truth. If implementation choices change, update the relevant decision record and tests.

Implementation sequence and engineering details

Implement in small, verifiable increments. Do not generate the entire codebase in one unreviewed change. Each phase must end with tests, a build/run check, updated documentation, and a concise report.

1. Request API

Accept query, tenant/user context, conversation context when enabled, requested response mode, and correlation ID. Validate payloads and apply rate limits.

Implementation checklist:

- Define typed request/response models and service interfaces before connecting UI components.
- Add migrations/indexes and explicit error handling for the data path.
- Add unit tests for normal, boundary, and failure cases.
- Add structured logging and a correlation ID without logging secrets or raw sensitive content.
- Document configuration, trade-offs, and operational behavior.

2. Policy and cache safety

Normalize query text; construct a cache key that includes tenant, authorization scope, embedding/model version, corpus version, prompt version, and relevant generation settings. Never share cached answers across incompatible security scopes.

Implementation checklist:

- Define typed request/response models and service interfaces before connecting UI components.
- Add migrations/indexes and explicit error handling for the data path.
- Add unit tests for normal, boundary, and failure cases.
- Add structured logging and a correlation ID without logging secrets or raw sensitive content.
- Document configuration, trade-offs, and operational behavior.

3. Semantic cache

Create query embeddings; retrieve candidate cached questions; use a configurable similarity threshold and optional exact-match fallback. Return a cache hit only when scope and freshness checks pass. Support TTL, invalidation, and cache bypass.

Implementation checklist:

- Define typed request/response models and service interfaces before connecting UI components.
- Add migrations/indexes and explicit error handling for the data path.
- Add unit tests for normal, boundary, and failure cases.
- Add structured logging and a correlation ID without logging secrets or raw sensitive content.
- Document configuration, trade-offs, and operational behavior.

4. Cost-aware router

Classify request complexity using configurable rules and/or a lightweight model. Select small/large model based on complexity, context size, required reasoning, confidence, latency target, and budget policy. Provide deterministic fallback when a provider fails.

Implementation checklist:

- Define typed request/response models and service interfaces before connecting UI components.
- Add migrations/indexes and explicit error handling for the data path.
- Add unit tests for normal, boundary, and failure cases.
- Add structured logging and a correlation ID without logging secrets or raw sensitive content.
- Document configuration, trade-offs, and operational behavior.

5. RAG pipeline

Retrieve relevant chunks, assemble bounded context, generate cited answers, and return source references. Keep retrieval and generation separately measurable.

Implementation checklist:

- Define typed request/response models and service interfaces before connecting UI components.
- Add migrations/indexes and explicit error handling for the data path.
- Add unit tests for normal, boundary, and failure cases.
- Add structured logging and a correlation ID without logging secrets or raw sensitive content.
- Document configuration, trade-offs, and operational behavior.

6. Usage ledger and dashboard

Record model/provider, token usage when available, estimated cost, cache status, retrieval/generation latency, total latency, route reason, quality signals, and errors. Label estimates separately from provider-reported billing.

Implementation checklist:

- Define typed request/response models and service interfaces before connecting UI components.
- Add migrations/indexes and explicit error handling for the data path.
- Add unit tests for normal, boundary, and failure cases.
- Add structured logging and a correlation ID without logging secrets or raw sensitive content.
- Document configuration, trade-offs, and operational behavior.

7. Evaluation and operations

Run a fixed benchmark of representative questions, compare cache/routing variants against a baseline, track quality regressions, and support configuration changes without code edits.

Implementation checklist:

- Define typed request/response models and service interfaces before connecting UI components.
- Add migrations/indexes and explicit error handling for the data path.
- Add unit tests for normal, boundary, and failure cases.
- Add structured logging and a correlation ID without logging secrets or raw sensitive content.
- Document configuration, trade-offs, and operational behavior.

Detailed request flow

1. Authenticate request and establish tenant/security scope; validate input and create request ID.

2. Normalize query and calculate a cache-scope/version fingerprint.
3. Create query embedding and search eligible cache candidates; reject candidates failing scope, freshness, or threshold checks.
4. On a valid hit, return the stored answer/citations with cache metadata and record latency/cost avoided as an estimate, not as provider billing.
5. On a miss, retrieve authorized chunks, assess complexity, choose a model, generate a grounded answer, and capture token usage.
6. Write cache entry with TTL and all version/scope metadata; record route reason, timings, and cost estimate.
7. Return response and expose aggregate metrics to the dashboard.

Configuration and environment

- Provide .env.example with DATABASE_URL, REDIS_URL when used, model provider selection, model names, embedding model, API keys as blank placeholders, log level, and feature flags.
- Validate required settings at startup and fail with actionable errors that do not print secret values.
- Keep local defaults suitable for development only; document production secret management separately.
- Provide deterministic fake model and embedding adapters for CI tests so the core suite does not require paid API calls.

Test strategy

- Unit tests: normalization, policy decisions, metadata validation, ranking/fusion, error mapping, cost calculations, and schema validation.
- Integration tests: PostgreSQL migrations, vector retrieval, Redis/Neo4j interactions where applicable, and API workflows.
- Security tests: authorization boundaries, prompt injection handling, unsafe cache reuse, malformed payloads, and sensitive logging.
- Evaluation tests: fixed dataset, baseline comparison, documented metrics, and regression thresholds.
- Frontend tests: key dashboard states, validation, permission errors, empty states, and API error handling.
- Smoke test: clean Docker Compose startup, health checks, sample ingestion, one query, and dashboard data.

Deployment and operations

- Use Docker Compose for local development with health checks and persistent volumes where required.
- Include migration and seed commands; avoid implicit schema changes on every request.
- Set request timeouts and bounded retries; make ingestion jobs idempotent.
- Document backup/restore expectations, data deletion, cache invalidation, logs/metrics, and known scaling limits.
- Provide a demo script and troubleshooting section.